Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Project

Stoquify / AqStoqFlow enterprise management platform.

Workspace

`E:\ohada saas\Focused projects\stoquify`

Domain

Platform-wide workflow architecture, business-process assurance, and source-of-truth governance.

Mission

Inspect the system thoroughly and produce an evidence-backed inventory and assessment of every material business, administrative, financial, security, compliance, and technical workflow.

Determine whether each workflow:

- Produces the intended business result exactly and consistently.
- Uses an authoritative service-owned source of truth.
- Preserves tenant isolation, authorization, auditability, and data integrity.
- Avoids duplicated business rules, conflicting records, UI-owned truth, and fragmented state transitions.
- Is efficient, understandable, resilient, observable, and productive for its intended user.
- Integrates correctly with dependent workflows, accounting controls, evidence chains, reporting surfaces, and release gates.

Propose prioritized modifications, refactors, additions, consolidations, and security controls that move Stoquify toward becoming each tenant's authoritative source of:

- Operational truth.
- Financial and accounting truth.
- Inventory and commercial truth.
- Workforce and payroll truth.
- Compliance and statutory truth.
- Approval, audit, and control truth.

Do not implement broad remediation during this run. Complete the evidence-led audit, source-of-truth map, workflow registry, and prioritized remediation roadmap first.

Definition of the Ultimate Goal

Stoquify qualifies as a single source of truth only when:

1. Every material business fact has one clearly identified authoritative owner.
2. Mutations occur through controlled service boundaries rather than UI or duplicated helper logic.
3. Every record is tenant-scoped, permission-controlled, traceable, and recoverable.
4. Derived dashboards and reports expose provenance, freshness, and reconciliation status.
5. Corrections preserve history instead of silently overwriting evidence.
6. Financially relevant events reconcile to ledger, payment, inventory, payroll, or close evidence where applicable.
7. Offline operations, retries, imports, and integrations are idempotent and conflict-aware.
8. Different roles and interfaces receive consistent projections of the same underlying truth.
9. Workflow completion means the intended business outcome was achieved—not merely that a page, endpoint, or status exists.
10. Claims of readiness are backed by executable tests, evidence, and observed behavior.

Team and Agent Routing

Use a lead orchestrator to coordinate bounded specialist investigations.

Assign non-overlapping evidence areas to appropriate agents:

1. Architecture and dependency analyst
    - Domain boundaries, service ownership, dependency direction, duplicated implementations, event flow, and architectural drift.
2. Security and identity analyst
    - Authentication, tenant boundaries, RBAC, module entitlement, step-up authentication, maker-checker controls, redaction, abuse prevention, and audit evidence.
3. Business workflow analyst
    - Workflow triggers, actors, preconditions, decisions, state transitions, outcomes, exceptions, corrections, and cross-domain dependencies.
4. Data and source-of-truth analyst
    - Prisma models, canonical records, competing stores, derived projections, provenance, freshness, reconciliation, migration, and deletion semantics.
5. Finance and controls analyst
    - Ledger effects, payments, cash, reconciliation, close assurance, inventory valuation, payroll accounting, approval evidence, and financial control integrity.
6. UI/UX workflow analyst

    - Role-aware navigation, task completion, state visibility, feedback, recovery paths, accessibility, responsive behavior, and consistency between UI and service truth.
7. Reliability and release analyst
    - Idempotency, queues, offline replay, retries, error handling, observability, background workers, tests, runbooks, policy gates, and release evidence.

Use installed skills only when relevant and only after reading their `SKILL.md` files. Prefer the smallest set of skills that covers the investigation. Do not invent unavailable skills or allow multiple agents to audit the same surface without a deliberate cross-check.

Universal Operating Principles

- Begin with evidence, not assumptions.
- Read repository instructions before analyzing code.
- Preserve the dirty worktree and do not overwrite unrelated user changes.
- Use existing knowledge graphs for architectural and impact analysis.
- Keep business truth server-side and service-owned.
- Treat dashboards as projections, not authorities.
- Distinguish implemented, partial, simulated, mocked, stale, missing, blocked, and externally unverified behavior.
- Do not equate route presence with workflow completion.
- Do not self-certify security, statutory compliance, accounting correctness, or production readiness.
- Keep statutory and country-specific logic configurable, versioned, provenance-backed, and subject to expert review.
- Do not perform destructive database operations, resets, migrations, or broad refactors.
- Do not clean unrelated lint failures or modify unrelated modules.
- Record uncertainty, skipped evidence, and access limitations explicitly.

Evidence-First Discovery

Inspect evidence in this order:

1. Repository governance
    - `AGENTS.md`
    - relevant local instructions
    - current branch and worktree status
    - package scripts and existing verification commands
2. Existing plans and assurance evidence
    - `what-next/`
    - `innovation/`
    - `docs/`
    - architecture reports

    - readiness registers
    - execution ledgers
    - release reports
    - screenshots and browser evidence
    - known blockers and deferred decisions
3. Knowledge graphs
    - `graphify-out/graph_components.json`
    - `graphify-out/graph_actions.json`
    - `graphify-out/graph_app.json`
    - `graphify-out/graph_hooks.json`
    - `graphify-out/graph_types.json`
    - corresponding `GRAPH_REPORT_*.md` files
4. Runtime and application surfaces
    - `app/`
    - `app/api/`
    - `actions/`
    - `services/`
    - `components/`
    - `hooks/`
    - `lib/`
    - `config/`
    - `prisma/schema.prisma`
    - migrations and seeds
    - `scripts/`
    - focused unit, integration, browser, and release-gate tests

Cross-check graphs against the current source because graph artifacts may be stale.

Workflow Enumeration

Create a canonical registry covering at least the following workflow families where they exist:

1. Authentication, registration, invitation, session, and recovery.
2. Tenant creation, organization selection, membership, and organization switching.
3. Roles, permissions, module entitlements, subscriptions, and provisioning.
4. Organization settings, locations, terminals, registers, and master data.
5. Customers, suppliers, items, categories, pricing, tax, and catalogs.
6. Inventory receipt, transfer, count, adjustment, reversal, valuation, and reconciliation.
7. Purchasing, receiving, invoice matching, accounts payable, and supplier settlement.
8. POS sale, shift, cash drawer, receipt, refund, void, correction, and offline replay.
9. Payments, mobile money, provider events, statements, suspense, matching, and reconciliation.
10. Accounting events, journals, ledger posting, close, certification, and reporting.

11. HRIS identity, contracts, organization structure, compensation, attendance, leave, and employee lifecycle.
12. Payroll readiness, calculation, approval, declaration, payment, reconciliation, payslip, and accounting close.
13. Compliance, tax, statutory declarations, country packs, and evidence provenance.
14. Approvals, maker-checker decisions, step-up authentication, overrides, and exception handling.
15. Document, evidence, redaction, retention, export, and audit-pack workflows.
16. Reporting, dashboards, analytics, snapshots, read models, and freshness controls.
17. Notifications, errors, escalations, incidents, support, and recovery.
18. Import, migration, backfill, correction, archival, and deletion workflows.
19. Offline synchronization, replay, conflict resolution, queues, and background processing.
20. AI copilot, automation, recommendations, approvals, and human-review boundaries.
21. Release readiness, policy gates, observability, operational runbooks, and rollback.
22. Public and external workflows, including receipts, webhooks, APIs, integrations, and abuse controls.

Discover and add material workflows not represented by this initial taxonomy.

Workflow Record Contract

For every discovered workflow, capture:

- Workflow ID and name.
- Business objective and expected result.
- Domain and authoritative owner.
- Actors and roles.
- Trigger and entry points.
- Preconditions.
- UI routes and components.
- Actions, APIs, services, and workers.
- Prisma models and persisted records.
- State machine and allowed transitions.
- Approval and authorization rules.
- Module-entitlement requirements.
- Inputs, outputs, and downstream consumers.
- Events, side effects, and financial effects.
- Audit and evidence records.
- Redaction and privacy boundaries.
- Idempotency, retry, and concurrency behavior.
- Failure, compensation, correction, and recovery paths.
- Offline or external integration behavior.
- Observability and operator diagnostics.
- Tests and release evidence.
- Current implementation status.
- Known inconsistencies or duplicated truth.

- Confidence level and supporting evidence.

Evaluation Framework

Score each workflow from 0 to 5 for:

1. Completeness.
2. Business correctness and exactness.
3. Architectural consistency.
4. Source-of-truth integrity.
5. Security and tenant isolation.
6. RBAC and entitlement enforcement.
7. Auditability and evidence quality.
8. Efficiency and unnecessary work.
9. Resilience and recovery.
10. Idempotency and concurrency safety.
11. Observability and supportability.
12. UI/UX productivity.
13. Accessibility and responsive usability.
14. Integration consistency.
15. Test and release-evidence strength.

For every score:

- Cite concrete evidence.
- Explain the gap preventing a higher score.
- Assign a confidence level.
- Do not award a production-ready score based only on mocks, fixtures, reports, or code presence.

Required Analyses

1. Source-of-Truth Map

Identify:

- Canonical business entities.
- The service or subsystem owning each entity.
- Authorized mutation paths.
- Derived read models and dashboards.
- Competing or duplicated stores.
- UI-derived or client-owned business truth.
- Stale projections and missing freshness indicators.
- Records lacking provenance or reconciliation.
- Destructive updates that should be corrections or reversals.

- Cross-domain facts without a clear owner.

2. Workflow Consistency Analysis

Find:

- Different implementations of the same workflow.
- Inconsistent status vocabularies.
- Mismatched permissions between page, action, API, and service.
- Navigation options that do not match entitlements.
- Services bypassed by direct Prisma access.
- UI states that disagree with server truth.
- Reports that calculate differently from transactional services.
- Duplicate approval, error, audit, or notification patterns.
- Missing locale, accessibility, responsive, or recovery behavior.

3. Efficiency and Productivity Analysis

Find:

- Repeated manual entry.
- Unnecessary approvals or handoffs.
- Excessive page switching.
- N+1 reads and duplicated queries.
- Repeated recalculation of stable facts.
- Missing bulk operations.
- Missing task queues or operator workbenches.
- Background work performed synchronously.
- Workflows with unclear next actions.
- Reports that cannot drive corrective action.
- Automation opportunities with safe approval boundaries.

4. Security and Control Analysis

Evaluate:

- Authentication and session assurance.
- Tenant escape risks.
- Missing permissions or entitlement checks.
- Privileged and sensitive actions.
- Maker-checker requirements.
- Fresh-auth boundaries.
- Enumeration and abuse resistance.
- Sensitive logging and error leakage.
- Evidence access and export controls.

- Payroll, payment, fiscal, identity, and document redaction.
- Webhook authenticity and replay protection.
- Mutation idempotency and concurrency.
- Immutable audit requirements.
- Fraud and segregation-of-duties risks.

5. Outcome Exactness Analysis

For each high-risk workflow, trace at least one complete happy path and one failure or correction path from trigger to persisted outcome.

Verify that:

- The final record matches the intended business result.
- All dependent records are consistent.
- Financial and inventory consequences reconcile.
- The UI confirms the actual persisted result.
- Failures do not leave partial or misleading state.
- Retries do not duplicate effects.
- Corrections remain traceable.
- Reports and dashboards reflect the authoritative outcome.

Prioritization

Classify findings as:

- P0 — immediate security, financial-integrity, tenant-isolation, or irreversible data risk.
- P1 — material workflow failure, conflicting truth, missing control, or unreliable outcome.
- P2 — productivity, consistency, resilience, observability, or UX weakness.
- P3 — strategic enhancement or optimization.

For every proposed remediation include:

- Problem and evidence.
- Root cause.
- Recommended solution.
- Authoritative owner.
- Affected workflows and dependencies.
- Files or modules likely involved.
- Data or migration impact.
- Security and control impact.
- UI/UX impact.
- Testing requirements.
- Rollout and rollback strategy.
- Estimated effort and sequencing.
- Risks of not implementing.

- Acceptance criteria.

Separate:

- Immediate containment.
- Surgical correction.
- Structural refactor.
- Strategic addition.

Expected Artifacts

Save the audit under:

`what-next/workflow-assurance/`

Produce:

1. `STOQUIFY_PLATFORM_WORKFLOW_REGISTRY_2026-07-28.json`
    - Machine-readable workflow inventory and scores.
2. `STOQUIFY_PLATFORM_WORKFLOW_SYSTEM_AUDIT_2026-07-28.md`
    - Executive summary, methodology, findings, evidence, and domain assessments.
3. `STOQUIFY_PLATFORM_SOURCE_OF_TRUTH_MAP_2026-07-28.md`
    - Canonical owners, mutation paths, projections, duplication, provenance, and trust gaps.
4. `STOQUIFY_PLATFORM_WORKFLOW_REMEDIATION_ROADMAP_2026-07-28.md`
    - Prioritized remediation, dependencies, phases, acceptance criteria, and recommended next prompts.
5. `STOQUIFY_PLATFORM_WORKFLOW_INCONSISTENCY_REGISTER_2026-07-28.json`
    - Structured register of duplicated truth, inconsistent states, bypasses, and cross-domain conflicts.

The reports must clearly distinguish:

- Confirmed facts.
- Evidence-supported inferences.
- Unverified assumptions.
- Missing evidence.
- External dependencies.
- Blocked checks.

Verification Strategy

First inspect `package.json` and existing scripts. Run only commands relevant to the evidence being validated.

Candidate checks, when present and appropriate:

```
npm run typecheck
npm run prisma:validate
npm run policy:gates
npm run workflow:assurance:runtime-check
npm run module:surface:inventory
npm run build:app```

Also run focused tests for representative high-risk workflows.

For every command record:

- Exact command.
- Result: passed, failed, timed out, skipped, or blocked.
- Relevant output.
- Whether failure is caused by the audited workflow or unrelated repository debt.

# Risk Controls

- Do not reset, clean, or rewrite the dirty worktree.
- Do not modify production data.
- Do not run destructive Prisma commands.
- Do not broaden an audit finding into an unapproved implementation.
- Do not expose secrets or sensitive data in reports.
- Redact personal, payroll, payment, fiscal, and provider information.
- Preserve service ownership and domain boundaries.
- Preserve append-only evidence and correction history.
- Treat country-pack and statutory conclusions as requiring expert validation.
- Treat inaccessible external providers as unverified.
- Stop and request authorization before any destructive, externally visible, or materially scope-expanding action.

# Success Criteria

The audit is complete only when:

- Every material workflow family has been examined or explicitly marked absent or unverified.
- Every registered workflow identifies an expected business result and authoritative owner.
- High-risk workflows include end-to-end happy-path and failure/correction-path traces.
- Duplicate and conflicting sources of truth are enumerated.
- Permission, entitlement, tenant, audit, redaction, and recovery boundaries are evaluated.
- Findings are evidence-linked and confidence-rated.
- Recommendations are prioritized, dependency-ordered, and acceptance-testable.
- Reports distinguish current truth from proposals.
- Verification results and blockers are recorded honestly.
- The remediation roadmap shows a credible path toward one authoritative, traceable, secure platform truth without requiring a destabilizing rewrite.
```

```
# Non-Goals

- Do not implement the full remediation roadmap in this run.
- Do not refactor unrelated code.
- Do not redesign every dashboard.
- Do not replace working domain services without evidence.
- Do not create speculative modules unrelated to identified workflow gaps.
- Do not certify legal, statutory, accounting, or security compliance.
- Do not hide uncertainty or convert missing evidence into optimistic assumptions.

# Final Handoff

Conclude with:

- Executive verdict.
- Highest-risk inconsistencies.
- Strongest existing foundations.
- Top ten remediation priorities.
- Recommended implementation sequence.
- Commands run and their outcomes.
- Unresolved blockers.
- Three to five narrowly scoped follow-up prompts for the highest-value remediation slices.
```